คู่มือโครงงาน MealQ

ภาพรวมโครงงาน

MealQ เป็นเว็บแอปสั่งอาหารล่วงหน้าและจัดการคิวรับอาหารที่ร้าน ลูกค้าเลือกเมนู สั่งซื้อ ติดตามสถานะ และแสดงรหัสหรือ QR ให้ร้านตรวจเมื่อรับอาหาร ร้านค้าจัดการเมนู สูตร วัตถุดิบ และคิวผ่านระบบเดียวกัน ส่วนผู้ดูแลระบบจัดการผู้ใช้และอนุมัติร้านก่อนเปิดขาย

เอกสารนี้อธิบายเทคโนโลยีที่ใช้กับงานแต่ละส่วน โครงสร้างข้อมูล และวิธีติดตั้งจาก GitHub เพื่อพัฒนาและทดลองใช้งานในเครื่อง โดยใช้ฐานข้อมูลแยกจากระบบจริง

ปรับปรุงวันที่ 20 ตุลาคม 2569

Repository: https://github.com/MiniProject-Web-MealQ/food-ordering-platform-mongodb

ผู้ใช้และขอบเขตการทำงาน

บทบาท	งานหลัก
ลูกค้า customer	ค้นหาร้านและเมนู สุ่มอาหาร จัดตะกร้า เก็บและใช้คูปอง ติดตามออเดอร์ รีวิว และดู Wallet
เจ้าของร้าน merchant	จัดการข้อมูลและรูปของร้าน เวลาเปิดขาย เมนู ตัวเลือก สูตร วัตถุดิบ คิว และตรวจรหัสรับอาหาร
พนักงาน staff	ทำงานกับร้านที่แอดมินกำหนดให้ตามสิทธิ์ โดยไม่จัดการการแจ้งเตือนแทนเจ้าของร้าน
ผู้ดูแลระบบ admin	อนุมัติหรือระงับร้าน จัดการผู้ใช้ โปรโมชัน ออเดอร์ Wallet รายงาน และการตั้งค่าระบบ

สมัครด้วยตนเองได้เฉพาะลูกค้าหรือเจ้าของร้าน ร้านใหม่เริ่มในสถานะรออนุมัติ
พนักงานเป็นบัญชีที่ผูกกับร้านเดิมโดยแอดมิน ลูกค้าและร้านค้าเข้าสู่ระบบที่ /login ส่วนแอดมินใช้ /admin/login

ฟังก์ชันสำคัญ

- แค็ตตาล็อกร้านและเมนูใช้ข้อมูลจาก API รวมถึงรูป ราคา หมวดหมู่ และสถานะพร้อมขาย
- หน้าสุ่มอาหารใช้ตัวกรองและข้อมูลเมนูจริง พร้อมจำการตั้งค่าในเบราว์เซอร์เครื่องนั้น การสุ่มไม่ได้ใช้โมเดล AI
- ลูกค้าเก็บคูปองก่อนใช้งาน และเลือกได้หนึ่งใบต่อออเดอร์ ระบบคำนวณราคาและตรวจเงื่อนไขที่ Backend
- ตรวจวัตถุดิบทั้งตะกร้าก่อนสั่งซื้อ และตรวจซ้ำตอนสร้างออเดอร์เพื่อรองรับการซื้อพร้อมกัน
- ติดตามออเดอร์ผ่าน Socket.IO พร้อมการดึงข้อมูลสำรอง และเลือกแจ้งเตือนในแอป อีเมล หรือ Web Push ได้ตามการตั้งค่า
- ร้านตรวจรหัสรับอาหาร 4 หลักหรือ QR เพื่อจบออเดอร์ มีหน้าสาธิตขั้นตอนที่ /demo

ขอบเขตการชำระเงิน: ค่าเริ่มต้นเป็น Demo ทั้ง PromptPay และ Wallet
ไม่มีการโอนเงินจริงหรือเชื่อมธนาคาร เวลารออาหารเป็นค่าประมาณ และระบบเน้นรับอาหารที่ร้าน ไม่ได้จัดการขนส่งอาหาร

เทคโนโลยีและตำแหน่งที่นำมาใช้

เวอร์ชันหลักด้านล่างอ้างอิง dependency ของโครงงาน ให้ใช้ npm ci เพื่อติดตั้งเวอร์ชันตาม package-lock.json ของแต่ละส่วน

เทคโนโลยี	ใช้ในส่วนใด
HTML และ CSS	โครงสร้างฟอร์ม รายการอาหาร ตาราง และรูปแบบหน้าจอที่รองรับมือถือ
JavaScript	จัดการข้อมูลด้วย Array และ Object รับ Event ตรวจฟอร์ม คำนวณยอด และเรียก API แบบ async/await
React 19 และ React Router 8	แบ่ง UI เป็น component จัดการ state และเส้นทางของลูกค้า ร้านค้า และแอดมิน
Vite 8 และ Tailwind CSS 4	เปิด development server ส่งต่อ API ผ่าน proxy สร้างไฟล์ build และจัด responsive layout
Node.js และ Express 5	รัน Backend รับ HTTP request แยก routes และตรวจสิทธิ์ก่อนเข้า controller หรือ service
MongoDB และ Mongoose 9	เก็บ document กำหนด Schema ตรวจรูปแบบข้อมูล ใช้ ObjectId อ้างอิง และสร้างดัชนี
Fetch API และ JSON	ส่งข้อมูลระหว่างหน้าเว็บกับ API พร้อม session cookie และแสดงผลหรือข้อผิดพลาดที่ตอบกลับ
Socket.IO	แจ้งเหตุการณ์ออเดอร์และคิวระหว่างลูกค้ากับร้านโดยไม่ต้องรีเฟรชทั้งหน้า
Nodemailer	ส่ง OTP สมัครหรือเปลี่ยนอีเมล ลิงก์ตั้งรหัสผ่านใหม่ และอีเมลสถานะออเดอร์ผ่าน SMTP
Web Push และ Service Worker	แจ้งเตือนอุปกรณ์ที่อนุญาตไว้ แม้ไม่ได้เปิดแท็บ MealQ โดยขึ้นกับการรองรับของเบราว์เซอร์
qrcode และ jsqr	สร้าง QR รับอาหาร และอ่าน QR ผ่านกล้องฝั่งร้านค้า
Framer Motion	สร้าง animation ของส่วนติดต่อผู้ใช้ เช่น แถบตัวกรอง

ตัวอย่างการประยุกต์บทเรียน: map() ใช้แสดงรายการเมนู, filter() ใช้กรองอาหาร, reduce() ใช้รวมยอดตะกร้า, Event ใช้รับการแก้ฟอร์มและยืนยันออเดอร์ ส่วน HTTP/JSON เชื่อมหน้าเว็บเข้ากับ Node.js API โค้ด Frontend ใช้ ES Modules และ Backend ใช้ CommonJS Modules

สถาปัตยกรรมและโครงสร้างโค้ด

Browser แสดงผล React และเรียก Express API ผ่าน HTTP/JSON โดยไม่เชื่อมฐานข้อมูลโดยตรง Backend ตรวจ session และบทบาทผู้ใช้ ประมวลผลใน controller หรือ service แล้วใช้ Mongoose อ่านหรือเขียน MongoDB หลังเปลี่ยนสถานะจึงส่งเหตุการณ์ไปยังหน้าจอและระบบแจ้งเตือน

```
frontend/
  src/routes/       เส้นทางของหน้าเว็บ
  src/pages/        หน้าลูกค้า ร้านค้า และแอดมิน
  src/features/     state และตรรกะของแต่ละส่วน
  src/lib/          API client และ Socket.IO
  public/           รูปและ Service Worker
backend/
  src/server.js     เริ่ม Express และ Socket.IO
  src/routes/       กำหนด endpoint
  src/middleware/   ตรวจ session และสิทธิ์
  src/controllers/  รับ request และส่ง response
  src/services/     ราคา สต็อก ชำระเงิน และแจ้งเตือน
    src/models/         Mongoose Schema
  src/scripts/      seed ข้อมูลทดลอง
docs/               คู่มือและสไลด์นำเสนอ
docker-compose.yml  MongoDB สำหรับพัฒนาในเครื่อง
```

API ใช้ prefix /api/v1 ตัวอย่างที่สัมพันธ์กับการสั่งอาหาร:

Method และ endpoint	**หน้าที่**
GET /api/v1/restaurants	อ่านร้านอาหารที่แสดงต่อสาธารณะ
POST /api/v1/orders/check-stock	ตรวจสูตรและความพร้อมของวัตถุดิบ
POST /api/v1/orders	ตรวจข้อมูล คำนวณยอด และสร้างออเดอร์
PATCH /api/v1/merchant/orders/:id/status	ร้านเปลี่ยนสถานะออเดอร์
POST /api/v1/merchant/orders/:id/receive	ร้านตรวจรหัสหรือ QR เพื่อยืนยันรับอาหาร

Backend ใช้ session cookie ที่ลงลายเซ็น HMAC และมี HttpOnly รหัสผ่านเก็บเป็น hash ด้วย scrypt การตรวจบทบาทอยู่ฝั่งเซิร์ฟเวอร์ การซ่อนปุ่มในหน้าเว็บอย่างเดียวไม่ถือเป็นการตรวจสิทธิ์

ข้อมูลและขั้นตอนการสั่งอาหาร

ความสัมพันธ์ของข้อมูล

ข้อมูล	ความสัมพันธ์และเหตุผล
User และ Store	เก็บบัญชีและบทบาท เจ้าของร้านอ้างผ่าน owner_id ส่วนพนักงานใช้ store_id ของร้านที่ได้รับมอบหมาย
Store กับ Menu และ Ingredient	หนึ่งร้านมีหลายเมนูและวัตถุดิบ โดยอ้างร้านผ่าน store_id
Menu.ingredients	เก็บ ingredient_id ปริมาณต่อจาน และหน่วย หลายเมนูจึงใช้วัตถุดิบกองเดียวกันได้
Order และ Queue	Order อ้างลูกค้าและร้าน Queue มี order_id ไม่ซ้ำเพื่อเก็บคิวและรหัสรับอาหาร
Order.items	ฝังชื่อ ราคา จำนวน และตัวเลือก ณ เวลาสั่ง พร้อม menu_id เพื่อรักษาประวัติเมื่อเมนูเปลี่ยน
Wallet และ WalletTransaction	แยกยอดคงเหลือจากประวัติเติม จ่าย คืน และปรับยอด
Promotion และ CustomerCoupon	แยกเงื่อนไขโปรโมชันจากคูปองที่ลูกค้าแต่ละคนเก็บ
Notification และ EmailOutbox	เก็บข้อความในแอปและคิวส่งอีเมลสำหรับติดตามผลและส่งซ้ำเมื่อจำเป็น

Mongoose ref ใช้อ้างอิง document ในระดับแอป ไม่ใช่ foreign key
หรือการลบต่อเนื่องแบบฐานข้อมูลเชิงสัมพันธ์

จากเลือกอาหารจนรับอาหาร

1. ลูกค้าเลือกเมนู ตัวเลือก จำนวน และคูปองที่มีได้หนึ่งใบ
2. หน้าเว็บเรียกตรวจสต็อกก่อนยืนยันซื้อ Backend รวมความต้องการวัตถุดิบทั้งตะกร้าและตรวจหน่วยตามสูตร
3. ตอนสร้างออเดอร์ Backend ตรวจร้านเปิดและได้รับอนุมัติ เมนูพร้อมขาย สูตร สต็อก ราคา และคูปองอีกครั้ง
4. Backend จองวัตถุดิบด้วย conditional atomic update ก่อนหัก Wallet และสร้างคิว หน้าเว็บส่ง Idempotency-Key เพื่อป้องกันคำขอซ้ำเมื่อ retry
5. ร้านเปลี่ยนสถานะจาก pending ไป confirmed แล้ว preparing และ ready
   ลูกค้าได้รับการอัปเดตตามช่องทางที่เปิดไว้
6. เมื่อลูกค้ามารับ ร้านตรวจรหัส 4 หลักหรือ QR สำเร็จจึงเปลี่ยนเป็น completed
7. การยกเลิกที่อนุญาตจะคืนสต็อกและยอดที่จ่ายโดยป้องกันการคืนซ้ำ
   ลูกค้ายกเลิกเองได้ก่อนร้านเริ่มทำอาหาร

ระบบมี checkout recovery เพื่อจัดการขั้นตอนที่ค้างและชดเชยยอดหรือสต็อก ไม่ได้ใช้ MongoDB multi-document transaction จึงรันกับ MongoDB standalone ตาม Compose ใน repository ได้

สิ่งที่ต้องเตรียมก่อนติดตั้ง

- Git สำหรับ clone repository
- Node.js อย่างน้อย 22.22.0 ในสาย 22 หรือใช้สาย 24 พร้อม npm เนื่องจาก React Router ใน lock file ต้องการ Node.js ตั้งแต่ 22.22.0
- MongoDB ที่เชื่อมต่อได้ วิธีในคู่มือนี้ใช้ Docker Compose เปิด MongoDB 8.0 ในเครื่อง
- Docker และ Compose จำเป็นเฉพาะเมื่อเลือกใช้ MongoDB ผ่าน Docker บน Windows ให้เปิด Docker Desktop ก่อนรันคำสั่ง

ไม่ต้องติดตั้ง nodemon แบบ global และไม่มีคำสั่ง npm ที่โฟลเดอร์ราก ต้องติดตั้ง dependency ของ backend และ frontend แยกกัน

ตรวจเครื่องมือและดาวน์โหลดโค้ด

คำสั่งชุดนี้ใช้ได้ทั้ง PowerShell และ shell บน macOS/Linux สำหรับ PowerShell ให้เปลี่ยนเฉพาะ npm --version เป็น npm.cmd --version หากติด execution policy:

```
node --version
npm --version
git --version
docker compose version
git clone https://github.com/MiniProject-Web-MealQ/food-ordering-platform-mongodb.git
cd food-ordering-platform-mongodb
git switch develop
docker compose up -d mongo
docker compose ps
```

Compose ที่โฟลเดอร์รากเปิดเฉพาะ MongoDB และเก็บข้อมูลใน named volume โดยผูกพอร์ตกับ loopback เท่านั้น ถ้ามี MongoDB อยู่แล้วให้ข้าม Docker และแก้ MONGO_URI
ให้ตรงกับฐานข้อมูลสำหรับพัฒนาของตนเอง

ติดตั้งและรันในเครื่อง

Windows PowerShell

เปิด Terminal แรกจากโฟลเดอร์ repository เพื่อติดตั้ง Backend คัดลอกไฟล์ตัวอย่างเมื่อยังไม่มี .env และเพิ่มข้อมูลทดลอง:

```
cd backend
npm.cmd ci
if (!(Test-Path .env)) { Copy-Item .env.example .env }
npm.cmd run seed
npm.cmd run dev
```

เปิด Terminal ที่สองจากโฟลเดอร์ repository สำหรับ Frontend:

```
cd frontend
npm.cmd ci
if (!(Test-Path .env)) { Copy-Item .env.example .env }
npm.cmd run dev
```

ใช้ npm.cmd ใน PowerShell เพื่อเลี่ยงปัญหา execution policy ของสคริปต์ npm.ps1

macOS และ Linux

เปิด Terminal แรกจากโฟลเดอร์ repository:

```
cd backend
npm ci
[ -f .env ] || cp .env.example .env
npm run seed
npm run dev
```

เปิด Terminal ที่สองจากโฟลเดอร์ repository:

```
cd frontend
npm ci
[ -f .env ] || cp .env.example .env
npm run dev
```

Backend ต้องเริ่มจากโฟลเดอร์ backend เพื่อโหลด .env ถูกตำแหน่ง เมื่อแก้ค่าฝั่ง Backend ให้หยุดและเริ่ม process ใหม่ คำสั่งคัดลอกด้านบนไม่เขียนทับ .env ที่มีอยู่

ที่อยู่และพอร์ตเริ่มต้น

ส่วน	ค่าที่ใช้
หน้าเว็บ	http://localhost:5173 หรือ URL ที่ Vite แสดง
Backend	http://localhost:5000
ตรวจ API และฐานข้อมูล	http://localhost:5000/api/v1/health
MongoDB จาก Compose	mongodb://127.0.0.1:27018/mealq

ส่วน	ค่าที่ใช้
Frontend API	VITE_API_URL=/api/v1

ผลตรวจ health ควรมี status: "ok" และ database: "connected" ส่วน Vite ส่ง /api และ /socket.io ไป Backend พอร์ต 5000 หากเปลี่ยนพอร์ต Backend ต้องแก้ proxy ใน frontend/vite.config.js ด้วย

ค่าระบบและบริการเสริม

ไฟล์ backend/.env.example และ frontend/.env.example เป็นแม่แบบเท่านั้น เก็บค่าจริงใน .env ที่ Git ignore ไว้ ฝั่ง Frontend ใช้เฉพาะค่าที่เปิดเผยได้ ห้ามใส่ Client Secret หรือรหัส SMTP ในตัวแปร VITE

ตัวแปรฝั่ง **Backend**	หน้าที่
PORT	พอร์ต API ค่าเริ่มต้น 5000
MONGO_URI	ที่อยู่และชื่อฐานข้อมูลที่แอปใช้
FRONTEND_URL	Origin ของหน้าเว็บสำหรับ CORS และลิงก์ที่เกี่ยวข้อง
SESSION_SECRET	ความลับสำหรับลงลายเซ็น session เปลี่ยนเป็นค่าสุ่มเฉพาะระบบก่อนเผยแพร่
PAYMENT_MODE	demo เปิดการชำระและเติมยอดทดลอง ส่วน wallet ปิด PromptPay จำลองและการเติมยอดทดลองด้วยตนเอง
TRUST_PROXY_HOPS	จำนวน reverse proxy ที่เชื่อถือ ต้องตั้งตามเส้นทางจริงและไม่เปิด API ให้ข้าม proxy
GOOGLE_CLIENT_ID และ GOOGLE_CLIENT_SECRET	ข้อมูล OAuth ของ Google Login เป็นบริการเสริม
GOOGLE_REDIRECT_URI	callback ที่ต้องตรงกับ Google Console ทุกตัวอักษร
SMTP_HOST PORT SECURE USER PASSWORD FROM	ชุด SMTP_* สำหรับส่ง OTP ลิงก์ตั้งรหัสผ่าน และอีเมลออเดอร์

PAYMENT_MODE=wallet เป็น Wallet-only mode ยังไม่ได้เพิ่ม payment gateway หรือการรับเงินจริง อย่าตีความว่าเป็น production payment

Google Login

สร้าง OAuth client ชนิด Web application ใน Google Cloud กำหนด JavaScript origin เป็น http://localhost:5173 และ redirect URI เป็น http://localhost:5000/api/v1/auth/google/callback แล้วตั้งค่า GOOGLE_* ใน Backend ปุ่ม Google จะแสดงเมื่อมี Client ID และ Secret

ใช้ localhost ให้เหมือนกันตลอดการ login และ callback ไม่สลับกับ 127.0.0.1 หากใช้พอร์ต Frontend อื่นให้ปรับ origin และ FRONTEND_URL ตามจริง รายละเอียดเพิ่มเติมอยู่ที่ คู่มือ Google Login

SMTP และอีเมล

การสมัครด้วยรหัสผ่านต้องรับ OTP ผ่าน SMTP แม้ใน development ไม่มีรหัสข้ามขั้นตอน เมื่อยังไม่ตั้ง SMTP ให้ใช้บัญชี seed ทดลอง login ได้ก่อน สำหรับ Gmail ตัวอย่างของโครงงานใช้ smtp.gmail.com พอร์ต 587, SMTP_SECURE=false และ App Password โดยอีเมลผู้ส่งใน SMTP_FROM ตรงกับ SMTP_USER

ตรวจการเชื่อมต่อโดยไม่ส่งเมลจากโฟลเดอร์ Backend:

```
npm run mail:check
```

ผลตรวจการเชื่อมต่อผ่านยังไม่รับประกันว่าอีเมลเข้า Inbox ต้องทดสอบกับกล่องผู้รับด้วย อีเมลสถานะออเดอร์ต้องเปิดรับในหน้าตั้งค่าของลูกค้าหรือเจ้าของร้าน และใช้อีเมลที่ยืนยันแล้ว ระบบเริ่มส่งเหตุการณ์หลังเปิดใช้ ดู คู่มือ OTP ทางอีเมล

การแจ้งเตือนอุปกรณ์

Web Push ต้องมี HTTPS หรือ localhost, Service Worker และการกดอนุญาตของผู้ใช้ แอปสร้างและเก็บ VAPID keys ใน MongoDB อัตโนมัติ จึงไม่ต้องเพิ่ม key เองใน .env ลูกค้าเปิดจากเมนูโปรไฟล์ ส่วนเจ้าของร้านเปิดจากหน้าแจ้งเตือนร้าน

บน iPhone/iPad ให้ใช้ iOS/iPadOS 16.4 ขึ้นไป เพิ่ม MealQ ไปหน้าจอโฮม เปิดจากไอคอนนั้น แล้วกดอนุญาตแจ้งเตือน การส่งขณะปิดหน้าเว็บยังขึ้นกับระบบปฏิบัติการและการตั้งค่าของอุปกรณ์

ทดลองใช้งานหลังติดตั้ง

บัญชีจาก seed

บทบาท	อีเมล	รหัสผ่านเริ่มต้น
ลูกค้า	customer@mealq.local	Customer123!
ร้านค้า	merchant@mealq.local	Merchant123!
แอดมิน	admin@mealq.local	Admin123!

บัญชีเหล่านี้ใช้สำหรับฐานข้อมูลทดลองในเครื่อง ควรเปลี่ยนหรือลบบัญชีทดลองก่อนเปิดระบบสาธารณะ แอดมินใหม่อาจถูกนำไปเปลี่ยนรหัสเริ่มต้นก่อนใช้งานสิทธิ์ผู้ดูแล ใช้ browser profile แยกกันทดสอบลูกค้ากับร้าน เพราะ session cookie ของแต่ละ profile ใช้ร่วมกัน

npm run seed เพิ่มข้อมูลที่ยังขาดและคงรหัสผ่าน เมนู ร้าน และยอด Wallet เดิม ลูกค้าทดลองใหม่มี Wallet เริ่มต้น 1,000 บาทและโปรโมชัน LUNCH10 ชุด seed ไม่ใช่สำเนาฐานข้อมูลของ mealq.site และยังไม่เติมรูปหรือสูตรวัตถุดิบให้เมนู

ต้องตั้งสูตรก่อนทดสอบสั่งซื้อ: เมนู seed ที่ยังไม่มีสูตรจะถูกปฏิเสธด้วย RECIPE_NOT_CONFIGURED จนร้านผูกวัตถุดิบครบ การติดตั้งหรือ login สำเร็จเพียงอย่างเดียวยังไม่หมายความว่าเมนูพร้อมรับออเดอร์

เตรียมร้านและเมนูให้รับออเดอร์ได้

1. เข้าบัญชีร้านที่ /login แล้วเปิด /merchant/inventory เพิ่มวัตถุดิบ กำหนดหน่วย เช่น กรัม และยอดคงเหลือ เช่น 1,000

2. เปิดหน้าเมนูของร้าน แก้เมนู seed หรือสร้างเมนูใหม่ ระบุราคา หมวดหมู่ และสูตรต่อหนึ่งจาน เช่น ไก่ 100 กรัมกับข้าว 150 กรัม สูตรต้องมีอย่างน้อยหนึ่งวัตถุดิบ
3. ใช้หน่วยของสูตรให้ตรงกับวัตถุดิบ ตรวจว่าเมนูพร้อมขาย ร้านผ่านอนุมัติ และเปิดรับออเดอร์แล้ว ร้านที่สมัครใหม่ต้องให้แอดมินอนุมัติก่อน
4. เปิด profile เบราว์เซอร์อีกชุด เข้าบัญชีลูกค้า สั่งเมนูที่เตรียมไว้และชำระด้วย Wallet ทดลอง
5. ตรวจยอดวัตถุดิบหลังสร้างออเดอร์ จากนั้นฝั่งร้านรับออเดอร์ เริ่มทำ และตั้งพร้อมรับ
6. นำรหัสหรือ QR จากออเดอร์ของลูกค้าไปยืนยันฝั่งร้าน ตรวจว่าทั้งสองฝั่งแสดง completed

อย่าใช้ npm run seed -- --reset-demo เป็นคำสั่งติดตั้งปกติ เพราะจะคืนค่าข้อมูลและรหัสผ่านของบัญชี demo สำรองฐานข้อมูลก่อนใช้คำสั่งที่เปลี่ยนข้อมูลจำนวนมาก

ตรวจสอบและสร้างไฟล์สำหรับรัน

ตรวจโค้ด

จากโฟลเดอร์ Backend:

```
npm test
npm run check
```

จากโฟลเดอร์ Frontend:

```
npm test
npm run lint
npm run build
```

การทดสอบ workflow ที่ต้องใช้ MongoDB มี scripts เช่น test:integration, test:regression, test:otp, test:push, test:coupons และ test:order-notifications อ่านคำอธิบายใน README ก่อนรัน โดยใช้ฐานข้อมูลทดสอบแยกและให้มี MongoDB local พร้อมใช้งาน

ทดลองไฟล์ build ในเครื่อง

รัน npm run build ใน Frontend จากนั้นตั้ง FRONTEND_URL=http://localhost:5000 ใน backend/.env แล้วเริ่ม Backend ใหม่ด้วย npm start จากโฟลเดอร์ Backend เปิด http://localhost:5000 เพื่อให้ Express เสิร์ฟไฟล์ frontend/dist ไม่จำเป็นต้องเปิด Vite พร้อมกันสำหรับวิธีนี้ ต้องปรับ FRONTEND_URL แม้ใน development เพื่อให้ลิงก์อีเมลกลับมาที่หน้าเว็บที่เปิดอยู่ เมื่อกลับไปใช้ Vite ให้เปลี่ยนเป็น http://localhost:5173 และรีสตาร์ท Backend

สำหรับ HTTP local ให้ใช้ development ตามค่าเริ่มต้น หากตั้ง NODE_ENV=production ต้องมี SESSION_SECRET ใหม่และ HTTPS เพราะ session cookie ถูกกำหนดเป็น Secure ให้ตั้ง FRONTEND_URL เป็น public HTTPS origin ที่ผู้ใช้เปิดจริง ส่วน VITE_API_URL=/api/v1 ใช้ได้กับการเสิร์ฟ origin เดียวกันและเป็นค่าที่อ่านตอน build จึงต้อง build ใหม่เมื่อเปลี่ยน

Compose ที่ deploy/compose.yaml เป็นรูปแบบ deployment ของสภาพแวดล้อมที่เตรียม volume, network และไฟล์ config ไว้แล้ว ไม่ใช่คำสั่งเริ่มต้นสำหรับเครื่องใหม่ ให้เริ่มจาก Compose ที่รากของ repository ตามคู่มือนี้

ปัญหาที่พบบ่อย

อาการ	สิ่งที่ควรตรวจ
npm แจ้ง EBADENGINE	ตรวจ node --version ให้ผ่านขั้นต่ำ 22.22.0 และเริ่ม Terminal ใหม่หลังเปลี่ยน Node
npm หา package.json ไม่พบ	เข้า backend หรือ frontend ก่อนรัน npm โฟลเดอร์รากไม่มี package.json
Backend เชื่อมฐานข้อมูลไม่ได้	ตรวจ docker compose ps และ MONGO_URI โดย Compose ใช้พอร์ต host 27018 ไม่ใช่ 27017
หน้าเว็บเรียก API ไม่ได้	เปิด Backend พอร์ต 5000 ตรวจ VITE_API_URL และ proxy รวมถึง .env.local ที่อาจ override ค่าเดิม
สั่งซื้อไม่ได้เพราะสูตรหรือสต็อก	ผูกสูตรให้เมนู ตรวจหน่วย ปริมาณรวม และยอดคงเหลือใน Inventory
สมัครแล้วส่ง OTP ไม่ได้	ตั้ง SMTP_* ใหม่ รีสตาร์ท Backend และรัน mail:check ไม่มี OTP bypass
Google Login callback ไม่ตรง	ตรวจ origin, redirect URI, พอร์ต และการใช้ localhost ให้เหมือนกัน
เข้าแล้วหลุดหรือ cookie ไม่ถูกส่ง	ตรวจ HTTPS, NODE_ENV, FRONTEND_URL และไม่ปะปน localhost กับ 127.0.0.1
แจ้งเตือนไม่แสดงตอนปิดหน้าเว็บ	ตรวจสิทธิ์อุปกรณ์ หน้าแจ้งเตือนของบัญชี Service Worker และการตั้งค่าระบบปฏิบัติการ

หยุดแอป development ด้วย Ctrl+C ใน Terminal ที่รันอยู่ หากต้องหยุด MongoDB ให้ใช้ docker compose stop mongo ซึ่งเก็บ volume ไว้ อย่าใช้ docker compose down -v
เว้นแต่ตั้งใจลบข้อมูลและสำรองฐานข้อมูลเรียบร้อยแล้ว

การทำงานร่วมกันผ่าน Git

ก่อน commit ตรวจรายการเปลี่ยนแปลงและเลือกเพิ่มเฉพาะไฟล์ที่ต้องการเผยแพร่ เก็บ .env, OAuth secret, SMTP password, tunnel token และไฟล์สำรองฐานข้อมูลออกจาก Git เสมอ node_modules, dist และ output ถูก ignore ไว้แล้ว

```
git status --short
git diff --check
git add README.md docs/project-guide.md docs/MealQ-Project-Guide.docx
git commit -m "docs: document MealQ architecture and installation"
git push origin HEAD:develop
```

คำสั่ง push ตัวอย่างใช้กับทีมที่มีสิทธิ์แก้สาขา develop หาก repository กำหนดให้ใช้ pull request ให้ push สาขางานของตนแล้วเปิด pull request ตามกติกาทีม

ไฟล์อ้างอิงในโครงงาน

- dependency ของ Frontend และ dependency ของ Backend
- API server และ API client
- เส้นทางหน้าเว็บ และ Schema ฐานข้อมูล
- การสร้างออเดอร์ และ การจัดการวัตถุดิบ
- ตัวอย่างค่าระบบ Backend และ Frontend
- Compose สำหรับ MongoDB local และ สไลด์นำเสนอ